

No-Code Neural Network Text Classification for Customer Feedback: A Clothing Store Sentiment and Intent Classifier

Ram Kelly Jersen Leliza
College of Computer Studies and
Engineering
Jose Rizal University
Manila City, Philippines
ramkellyjersen.leliza@my.jru.edu

Abstract—This paper reports on a hands-on, no-code exercise in building a neural-network text classifier for customer feedback using the Machine Learning for Kids platform. A three-class model -- Positive_Feedback, Negative_Feedback, and Urgent_Support -- was first trained on customer messages for a clothing store domain. Testing on ten held-out phrases showed the model handled clear sentiment correctly, but assigned low, scattered confidence scores (3%-31%) to four phrases that did not belong to any of the three categories. A fourth class, General_Inquiry, was then added with ten new labeled examples and the model was retrained. The same four phrases were subsequently classified with 100% confidence, and overall accuracy across all ten test phrases rose to 90%. These results are used to discuss the practical differences between rule-based and learned text classification, the effect of dataset coverage on model confidence, and the stage of the pipeline at which feature extraction occurs.

Keywords—*neural networks, text classification, supervised learning, no-code machine learning, sentiment analysis, intent classification*

I. INTRODUCTION

Businesses that collect customer feedback at scale face a practical sorting problem: incoming messages must be routed to the right response -- a thank-you, a complaint-handling process, or an urgent escalation -- often within minutes. A traditional software approach to this problem would rely on explicit IF/ELSE rules keyed to specific words or phrases. A neural network takes a different approach: rather than being told which words mean what, it is shown many labeled examples and learns the statistical associations between word patterns and categories on its own.

This paper documents an exercise in which such a classifier was built without writing any code, using the Machine Learning for Kids platform, for a hypothetical clothing store. Three intent classes were defined -- Positive_Feedback (praise), Negative_Feedback (complaints), and Urgent_Support (urgent account or order issues) -- and a fourth class, General_Inquiry, was added midway through the exercise in response to observed model failures. The remainder of this paper describes the methodology used to train and test the model (Section II), the accuracy and confidence results obtained (Section III), a discussion connecting these results to the exercise's guide

questions (Section IV), and concluding observations (Section V).

II. METHODOLOGY

A. Tool and Platform

The Machine Learning for Kids platform (machinelearningforkids.co.uk) was used, working through its three built-in stages: Train, where tagged text examples are collected; Learn & Test, where the platform trains a neural network model on those examples and reports its status; and Make, where new, unseen text can be submitted to the trained model to see its predicted class and confidence score. At no point in this pipeline is a keyword-matching rule written by hand -- the platform converts the collected text into numerical features and lets the network learn the weighted associations between those features and the labels provided in Train.

B. Dataset

The chosen business domain was an online clothing store. Eleven training examples were written for each of the three original classes, covering realistic customer messages about fit, fabric quality, shipping delays, refunds, and account or payment problems. Positive_Feedback examples included messages such as praise for fit and fast shipping; Negative_Feedback examples included complaints about shrinking fabric, broken zippers, and incorrect sizing; Urgent_Support examples included duplicate charges, failed payments, and lost or delayed packages needed urgently.

C. Testing Procedure

A held-out test set of ten phrases, none of which appeared in training, was constructed following the three test categories specified in the exercise: clear-pattern cases expected to be classified with high confidence; ambiguous or sarcastic cases using positive-sounding words ("great," "fantastic") inside what are actually complaints, intended to probe whether the model relies on single trigger words; and out-of-vocabulary cases -- general shopping questions about sizing, store hours, discount codes, and international shipping -- that do not correspond to any of the three original classes.

D. Refinement Iteration

After the first round of testing, the four out-of-vocabulary phrases were consistently misclassified with low, scattered confidence scores (3%-31%), since the model had no class that actually matched their content and was forced to choose among the three it had been given. A fourth class, *General_Inquiry*, was created with ten new labeled examples covering sizing, store hours, discount, and shipping questions. The model was retrained, and the same ten test phrases from Section II-C were re-submitted to measure the change in accuracy and confidence.

III. RESULTS

Before the fourth class was added, the model correctly classified the six core *Positive_Feedback*, *Negative_Feedback*, and *Urgent_Support* test phrases, including both sarcastic phrases, but misclassified all four general-inquiry phrases with low confidence (31%, 4%, 3%, and 7%). After retraining with the *General_Inquiry* class added, all four of those phrases were classified correctly at 100% confidence. Table I summarizes the final per-category and overall accuracy across all ten test phrases.

TABLE I. Per-Category Accuracy After Refinement (n = 10 test phrases)

Category	Expected	Correct	Accuracy
Positive_Feedback	1	1	100%
Negative_Feedback	4	3	75%
Urgent_Support	1	1	100%
General_Inquiry	4	4	100%
Overall	10	9	90%

The single remaining error involved the phrase “Wow, love how my package still hasn’t arrived after three weeks,” expected as *Negative_Feedback* but classified as *Urgent_Support* at 79% confidence -- a plausible alternate label, since a three-week-late package is arguably both a complaint and an urgent shipping problem.

IV. DISCUSSION

A. Explicit Rules vs. Tagged Learning

Writing IF/ELSE rules for every possible customer complaint would require anticipating every way a customer might phrase the same underlying intent, plus handling negation, sarcasm, and new vocabulary customers invent over time. The two sarcastic test phrases in this exercise -- “Oh great, another shirt that shrank in the wash” and “Fantastic, the zipper broke on day one” -- both contain positive-sounding trigger words (“great,” “fantastic”) that a simple keyword rule would very likely route to *Positive_Feedback*. The trained model, however, classified both correctly as complaints, because it was not matching on a single keyword but on the pattern formed by the whole sentence. Reproducing that same sensitivity with hand-written rules would require an ever-growing, difficult-to-maintain set of exceptions and priority orderings -- exactly the brittleness that tagged learning is designed to avoid.

B. Data Bias and Dataset Quality

Before *General_Inquiry* existed, the model had no class whose examples resembled the four out-of-vocabulary phrases, so it was forced to force-fit each one into *Positive_Feedback*, *Negative_Feedback*, or *Urgent_Support*, producing very low and scattered confidence scores (3%-31%) rather than a single wrong answer with high confidence. This matters because a network's confidence score reflects how closely an input's features match patterns it has actually seen during training; when no class contains a genuine match, the output is effectively a weak guess split across classes. Adding just ten representative, topically diverse examples -- covering sizing, store hours, discounts, and shipping -- was enough to shift confidence on those same four phrases to 100%. This demonstrates that accuracy and confidence depend less on the total volume of training data than on whether that data actually represents the range of inputs the model will be asked to classify; the one remaining misclassification in Table I (a phrase that legitimately overlaps two categories) further shows that even a well-covered dataset cannot fully eliminate ambiguity, only reduce it.

C. Connecting to the Slide Architecture

Feature extraction is estimated to occur during the Learn & Test (training) stage of the pipeline, specifically at the moment the raw labeled text collected in Train is converted into numerical token or word-based features, immediately before the network computes the weighted associations between those features and each label. The Train stage itself only collects raw text and labels -- no numerical transformation happens there. The Make stage, in turn, reuses the same feature-extraction scheme learned during training to convert new, unseen input into the same numerical representation so it can be scored against the already-learned weights; it does not learn any new features of its own. The pipeline can therefore be summarized as: raw text collection (Train) → feature extraction and weight learning (Learn & Test) → feature extraction of new input and classification against learned weights (Make).

V. CONCLUSION

This exercise showed, with a concrete before-and-after comparison, that a neural network's accuracy and confidence are driven less by rule design and more by how well its training data represents the inputs it will face: adding one well-populated class raised accuracy on affected phrases from 0% to 100%. It also showed a clear practical advantage of learned pattern recognition over hand-coded rules on sarcastic phrasing that a keyword-based system would likely misroute. Future refinement of this model should add further sarcastic and negated examples to *Negative_Feedback* and additional boundary examples to disambiguate cases that legitimately overlap *Negative_Feedback* and *Urgent_Support*, as observed in Table I.

REFERENCES

- [1] Machine Learning for Kids, “Recognizing Text,” IBM/Dale Lane. [Online]. Available: <https://machinelearningforkids.co.uk/>. Accessed: Jul. 27, 2026.
- [2] R. C. Raga Jr., “Exercise PT-P1: Deep Learning (Neural Networks) -- No-Code Text Classification using Neural Networks,” Lab Exercise Handout, 2026.